
Anteproyecto — Proyecto Final de Sistemas Operativos

Facultad de Ingeniería · Universidad de Antioquia · 2026

Título del proyecto	Servidor Web Concurrente con Políticas de Scheduling: Implementación y Evaluación de Rendimiento
Miembros del equipo	Samuel Puerta Patiño – CC 1023624795
Curso	Sistemas Operativos y Laboratorio

Resumen

Este proyecto propone implementar un servidor web concurrente en C sobre Linux, partiendo del código base monohilo provisto por el repositorio OSTEP (ostep-projects). El objetivo principal es transformar dicho servidor en un sistema multi-hilo mediante la biblioteca POSIX pthreads, implementando el patrón productor-consumidor con un buffer compartido y dos políticas de scheduling: FIFO (First In, First Out) y SFF (Smallest File First). El proyecto incluye el diseño y ejecución de experimentos estadísticos para comparar el rendimiento de ambas políticas bajo diferentes condiciones de carga, midiendo tiempo de respuesta, throughput y latencia. El resultado esperado es un servidor funcional, correctamente sincronizado y con resultados de experimentación reproducibles y verificables.

Introducción

Necesidad y problema que aborda el desafío

Los servidores web modernos deben atender miles de solicitudes simultáneas de forma eficiente. Un servidor monohilo procesa una solicitud a la vez; mientras sirve un archivo, todas las demás peticiones quedan bloqueadas esperando. Esto genera latencias inaceptables bajo carga concurrente y desperdicia la capacidad de los procesadores multi-núcleo disponibles en cualquier sistema actual.

El desafío central consiste en transformar un servidor web de un solo hilo en uno capaz de gestionar múltiples solicitudes en paralelo, garantizando acceso seguro a recursos compartidos (el buffer de solicitudes) y evaluando el impacto de distintas estrategias de scheduling en el rendimiento general del sistema.

Importancia en el contexto tecnológico actual

La concurrencia es uno de los pilares fundamentales de los sistemas operativos modernos y una competencia directamente requerida en el desarrollo de software de backend. Tecnologías como Nginx, Apache, Node.js y frameworks de alto rendimiento basan su arquitectura en los mismos principios que se explorarán en este proyecto: manejo de hilos, sincronización con mutexes y variables de condición, y scheduling de tareas.

Marco Teórico

Aspectos teóricos necesarios

Los conceptos fundamentales para comprender e implementar este proyecto son:

- Hilos (threads) y el modelo de memoria compartida: a diferencia de los procesos, los hilos dentro de un mismo proceso comparten el espacio de direcciones. Esto permite comunicación eficiente, pero introduce la necesidad de sincronización explícita para evitar condiciones de carrera.

-
- POSIX pthreads (pthread.h): la biblioteca estándar de hilos en Linux. Provee las primitivas `pthread_create()`, `pthread_join()`, `pthread_mutex_lock/unlock()` y `pthread_cond_wait/signal()`, que son las herramientas fundamentales para construir el pool de threads y el buffer sincronizado.
 - Patrón productor-consumidor: el hilo principal (productor) acepta conexiones HTTP entrantes y las coloca en un buffer compartido de capacidad fija. Los worker threads (consumidores) toman solicitudes del buffer y las atienden. El buffer debe protegerse con un mutex para garantizar exclusión mutua, y con variables de condición para suspender hilos cuando el buffer está lleno o vacío.
 - Scheduling FIFO (First In, First Out): política en la que el worker atiende siempre la solicitud más antigua del buffer. Garantiza equidad y evita inanición, pero no distingue por complejidad de la tarea.
 - Scheduling SFF (Smallest File First): el worker selecciona la solicitud cuyo archivo asociado tiene menor tamaño. Aproxima el algoritmo SJF (Shortest Job First) usando el tamaño del archivo como estimador del tiempo de servicio; minimiza el tiempo de respuesta promedio, pero puede generar inanición en archivos grandes.
 - Sección crítica y condiciones de carrera: fragmento de código que accede a recursos compartidos y que debe ejecutarse de forma atómica. Si dos hilos acceden simultáneamente sin sincronización, el resultado es indeterminado.

Relación con los temas del curso de Sistemas Operativos

El proyecto integra directamente los tres pilares del curso tal como los define el libro OSTEP (Operating Systems: Three Easy Pieces):

- Virtualización (procesos y memoria): el servidor es un proceso Linux que crea hilos; comprender la abstracción de proceso, espacio de direcciones y cómo el SO gestiona múltiples flujos de ejecución es el punto de partida.
- Concurrencia: es el núcleo del proyecto. Los temas de hilos, mutexes, variables de condición, problemas clásicos (productor-consumidor, lector-escritor) y debugging de condiciones de carrera se aplican directamente en la implementación.
- Persistencia (E/S y sistemas de archivos): el servidor sirve archivos del sistema de archivos local. La política SFF requiere consultar el tamaño del archivo antes de decidir qué solicitud atender, lo que involucra operaciones de `stat()` sobre el sistema de archivos.

Objetivos

Objetivo principal

Implementar un servidor web concurrente multi-hilo en C sobre Linux que gestione solicitudes HTTP mediante un pool de threads y un buffer sincronizado, con soporte para las políticas de scheduling FIFO y SFF, y evaluar estadísticamente su rendimiento bajo distintas condiciones de carga.

Objetivos específicos

1. Comprender y documentar la arquitectura del servidor monohilo base (`wserver.c`) provisto por el repositorio `ostep-projects`.
2. Implementar un pool de N threads workers utilizando pthreads, con un buffer compartido de capacidad M protegido por mutex y variables de condición.
3. Implementar la política de scheduling FIFO: el worker selecciona siempre la solicitud más antigua en el buffer.

-
4. Implementar la política de scheduling SFF: el worker selecciona la solicitud cuyo archivo asociado tiene el menor tamaño en disco.
 5. Diseñar y ejecutar un protocolo de experimentación con herramientas de generación de carga HTTP que permita comparar estadísticamente ambas políticas en escenarios de carga distintos.
 6. Analizar e interpretar los resultados en términos de throughput, tiempo de respuesta promedio, percentiles de latencia y variabilidad, reportando conclusiones verificables y reproducibles.

Metodología

Herramientas principales

- Lenguaje: C (C99/C11) con la biblioteca POSIX pthreads para manejo de hilos.
- Compilador: GCC con flags -Wall -Wextra -pthread -fsanitize=thread (ThreadSanitizer para detectar race conditions en desarrollo).
- Código base: repositorio ostep-projects (wserver.c, request.c, io_helper.c, io_helper.h).
- Generador de carga HTTP: httpperf o wrk para simular múltiples clientes concurrentes con distintos patrones de solicitudes.
- Análisis estadístico: Python con pandas, scipy y matplotlib para procesar los resultados experimentales y generar gráficas comparativas.
- Control de versiones: Git / GitHub para gestionar el código y la documentación.
- Entorno: Linux (Ubuntu 22.04 LTS).

Actividades en orden lógico

1. Leer y entender el código base monohilo de wserver.c, identificando el ciclo de aceptación de conexiones, el procesamiento de peticiones y las funciones de io_helper.
2. Estudiar el uso de pthreads: creación de hilos, pthread_mutex_t y pthread_cond_t a través de los capítulos de concurrencia del libro OSTEP.
3. Implementar el buffer compartido como arreglo circular o lista, con operaciones put() y get() protegidas por mutex y variables de condición para full/empty.
4. Implementar el pool de threads: N workers que esperan en get() hasta que haya trabajo disponible en el buffer.
5. Modificar el hilo principal para que, en lugar de atender la petición directamente, la deposite en el buffer mediante put().
6. Implementar la política FIFO: el buffer opera como cola FIFO y get() retorna siempre el elemento más antiguo.
7. Implementar la política SFF: get() recorre el buffer y retorna la petición cuyo archivo tiene menor tamaño (usando stat()).
8. Ejecutar pruebas funcionales básicas con curl y un navegador para verificar que el servidor responde correctamente bajo ambas políticas.
9. Diseñar los experimentos: definir variables independientes (política, número de threads N, tamaño de buffer M, patrón de solicitudes), variables de respuesta (throughput, latencia media, p95, p99) y parámetros estadísticos (número de réplicas, nivel de confianza 95%).
10. Ejecutar los experimentos con httpperf/wrk y recolectar datos.
11. Analizar los datos con Python: pruebas t de Student o Mann-Whitney U para comparar las políticas, y visualización con gráficas de caja y barras.

Diseño de Experimentos

Hipótesis

La política SFF reducirá el tiempo de respuesta promedio en escenarios con alta variabilidad en el tamaño de los archivos servidos, en comparación con FIFO, al priorizar las solicitudes de menor coste computacional. Sin embargo, se espera que FIFO ofrezca menor varianza en tiempos de respuesta en escenarios con archivos de tamaño homogéneo.

Variables y parámetros experimentales

- Variables independientes: política de scheduling (FIFO vs SFF), número de threads $N \in \{1, 2, 4, 8\}$, tamaño del buffer $M \in \{4, 8, 16\}$, distribución del tamaño de archivos (homogéneo vs. heterogéneo).
- Variables de respuesta: throughput (req/s), tiempo de respuesta promedio (ms), percentiles p50/p95/p99 de latencia, tasa de errores.
- Parámetros estadísticos: 30 réplicas por configuración, nivel de confianza 95%, prueba de hipótesis t de Student o Mann-Whitney U según normalidad de los datos (verificada con Shapiro-Wilk).

Protocolo de validación

- Escenario 1 — carga homogénea: todos los archivos del mismo tamaño (~10 KB); se espera rendimiento similar entre FIFO y SFF.
- Escenario 2 — carga heterogénea: mezcla de archivos pequeños (1 KB) y grandes (1 MB) en proporción 80/20; se espera ventaja de SFF en latencia media.
- Escenario 3 — carga alta: Clientes concurrentes; se evalúa saturación del servidor y comportamiento bajo presión.
- Las mediciones se realizarán con `httpperf --num-conns 1000 --rate 50` registrando las métricas en archivos CSV para posterior análisis en Python.

Se usarán gráficas de caja (boxplots) para visualizar la distribución de latencias y tablas con intervalos de confianza para el throughput. Las fuentes de error consideradas incluyen variabilidad del SO (scheduling del kernel), estado de la caché del disco y condiciones de red local.

Cronograma

El proyecto se desarrolla entre el 26 de marzo y el 13 de junio de 2026, con los siguientes hitos principales: entrega de propuesta (26 de marzo), presentación de avances (8 de mayo) y presentación final con informe (10-13 de junio).

Actividad	Sem 1 (Mar 26)	Sem 4 (Abr 14)	Sem 6 (Abr 28)	Sem 8 (May 8)	Sem 10 (May 22)	Sem 12 (Jun 10)
Lectura y análisis del código base (wserver.c)	•					
Estudio de pthreads y sincronización en C	•	•				
Implementación del pool de threads		•	•			
Implementación del buffer productor-consumidor		•	•			
Política de scheduling FIFO			•			
Política de scheduling SFF			•	•		

Presentación de avances (8 mayo)						
Diseño de experimentos y pruebas de carga						
Recopilación y análisis estadístico de resultados						
Redacción informe técnico (IEEE)						
Presentación final (10-13 junio)						

Referencias

1. R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, versión 1.10. Arpaci-Dusseau Books, 2023. [En línea]. Disponible en: <https://pages.cs.wisc.edu/~remzi/OSTEP/>
2. R. H. Arpaci-Dusseau, ostep-projects: Projects for an Operating Systems class. GitHub. [En línea]. Disponible en: <https://github.com/remzi-arpacidusseau/ostep-projects>
3. The Open Group, "pthread.h — threads", POSIX.1-2017 (IEEE Std 1003.1-2017). [En línea]. Disponible en: <https://pubs.opengroup.org/onlinepubs/9699919799/>